Ασφάλεια Πληροφοριακών Συστημάτων

Εργασία

Μπαρμπαγιάννος Βασίλειος

vmparmpg@ece.auth.gr

ΑΕΜ: 10685

---

**Προβλήματα Ασφαλείας**

Η εφαρμογή είναι ευάλωτη στα κάτωθι προβλήματα ασφαλείας.

1. SQL injection: τα δεδομένα που εισάγει ο χρήστης στη φόρμα login δεν ελέγχονται. Ως εκ τούτου, ένας επιτιθέμενος μπορεί να παρακάμψει τον έλεγχο ταυτότητας βάζοντας ειδικούς χαρακτήρες, π.χ. ‘ OR ‘1’=’1. Έτσι κάνει login χωρίς κωδικό.
2. Cross Site Scripting (XSS): Στη σελίδα notes.php τα δεδομένα που εισάγονται δεν καθαρίζονται. Αυτό συνεπάγεται ότι κάποιος μπορεί να γράψει κώδικα Javascript αντί για απλή σημείωση. Αυτός ο κώδικας θα τρέξει στον browser όποιου άλλου χρήστη δει τη σημείωση, επιτρέποντας την υποκλοπή cookies. Τα cookies που υποκλέπτονται μπορεί να τα χρησιμοποιήσει ο κακόβουλος για να υποδυθεί τον χρήστη, δηλαδή το θύμα υποκλοπής.
3. Οι κωδικοί πρόσβασης αποθηκεύονται στη βάση ως απλό κείμενο, χωρίς κρυπτογράφηση. Αυτό σημαίνει ότι όποιος αποκτήσει πρόσβαση στη βάση (έστω και σε λειτουργία read-only) θα μπορεί να δει – και να κλέψει – τους κωδικούς.
4. Η εφαρμογή συνδέεται στη βάση με δικαιώματα διαχειριστή. Έτσι όποιος κάνει SQL injection μπορεί να τη διαγράψει ολόκληρη (DROP table).
5. Χρησιμοποιείται το μη ασφαλές πρωτόκολλο HTTP, το οποίο δεν κρυπτογραφεί τα δεδομένα. Έτσι όποιος παρακολουθεί την κίνηση στο δίκτυο (network sniffing, π.χ. με το λογισμικό Wireshark) μπορεί να δει και να κλέψει τον κωδικό πρόσβασης.

**Τρόποι Αντιμετώπισης**

Για την αντιμετώπιση των κενών ασφαλείας που εντοπίστηκαν, εφαρμόστηκαν οι ακόλουθες βελτιώσεις στα αρχεία .php της εφαρμογής.

1. Αντιμετώπιση SQL injection:

Η αρχική υλοποίηση επέτρεπε τη συνένωση συμβολοσειρών για τη δημιουργία ερωτημάτων που «έσπαγαν» τη λογική του ερωτήματος, επιτρέποντας το injection.
Αντικαταστάθηκαν όλα τα ερωτήματα SQL με prepared statements.
Χρησιμοποιήθηκε η εντολή prepare() καθιστώντας αδύνατη την εκτέλεση κακόβουλου κώδικα SQL.
Διορθώθηκαν τα αρχεία: login.php, register.php, dashboard.php, notes.php.

2. Αντιμετώπιση XSS:

Τα δεδομένα που εισήγαγε ένας χρήστης στις σημειώσεις εμφανίζονταν κατευθείαν στον browser χωρίς έλεγχο. Έτσι μπορούσε κάποιος να εκτελέσει Javascript κώδικα.
Για το λόγο αυτό χρησιμοποιήθηκε κωδικοποίηση εξόδου.
Πριν την εμφάνιση οποιουδήποτε δεδομένου, χρησιμοποιείται η συνάρτηση htmlspecialchars($string, ENT_QUOTES, 'UTF-8').
Αυτή η συνάρτηση μετατρέπει τους ειδικούς χαρακτήρες σε HTML entities με αποτέλεσμα ο browser να τους ερμηνεύει σαν κείμενο και όχι ως κώδικα.
Αρχεία που διορθώθηκαν: notes.php, dashboard.php.

3. Αποθήκευση κρυπτογραφημένων κωδικών:

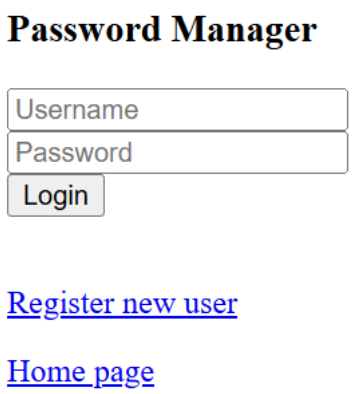
Οι κωδικοί αποθηκεύονταν ως απλό κείμενο (plain text), δίχως καμία κρυπτογράφηση. Οποιοσδήποτε είχε πρόσβαση στη βάση μπορούσε να τους δει. Τώρα βάλαμε κρυπτογράφηση, ώστε ο κωδικός να αποθηκεύεται κρυπτογραφημένος.
Κατά την εγγραφή (αρχείο register.php) χρησιμοποιείται η συνάρτηση password_hash($password, PASSWORD_DEFAULT), η οποία κρυπτογραφεί το password με τον αλγόριθμο bcrypt και προσθέτει τυχαίο αλάτι.
Κατά την είσοδο (αρχείο login.php) χρησιμοποιείται η συνάρτηση password_verify(), η οποία συγκρίνει τον κωδικό με το hash του αποθηκευμένου κωδικού.

**Επίθεση – Άμυνα**

Θα λειτουργήσουμε την εφαρμογή σε δύο φάσεις. Πρώτα θα χρησιμοποιήσουμε τους κώδικες όπως μας δόθηκαν, δηλαδή με τα κενά ασφαλείας για να δείξουμε τις τρύπες. Έπειτα θα χρησιμοποιήσουμε τους διορθωμένους κώδικες, αυτούς με τα χαρακτηριστικά ασφαλείας, για να δείξουμε ότι οι τρύπες διορθώθηκαν.

**Φάση Επίθεσης**

Βρίσκομαι στο login page.




**1. Επίθεση SQL injection.**

Πρώτα κάνω register έναν τυχαίο χρήστη «victim» με κωδικό «123456».

## New user registration

Successful registration!

You can now use the login page

Παρατηρώ ότι καταχωρήθηκε ο χρήστης victim, 123456 μέσα στη βάση μου. Οι κωδικοί είναι απροστάτευτοι, χωρίς κρυπτογράφηση.

| ←T→ | | | id | username | password |
|---|---|---|---|---|---|
| Επεξεργασία | Αντιγραφή | Διαγραφή | 1 | u1 | p1 |
| Επεξεργασία | Αντιγραφή | Διαγραφή | 2 | victim | 123456 |

*Απροστάτευτοι κωδικοί (δίχως κρυπτογράφηση).*

Στο username πληκτρολογώ ‘ OR ‘1’=’1’ # και στο password βάζω ό,τι να ‘ναι.

**Password Manager**

' OR '1'='1' #

••••••

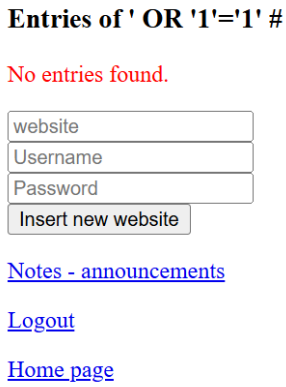
Login

Invalid username or password

Register new user

Home page

Πατάω Login και μπαίνω εδώ, στο Dashboard. Παράκαμψα τον έλεγχο. Μπήκα χωρίς να ξέρω το username και το password κανενός χρήστη.




*Επιτυχής παράκαμψη αυθεντικοποίησης μέσω SQL Injection. Η είσοδος επετράπη χωρίς έγκυρο κωδικό.*

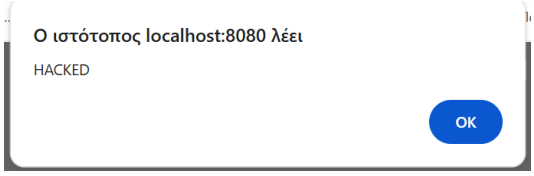
Λέει “No entries found.” διότι κανένας χρήστης στη βάση δεν έχει όνομα ‘ OR ‘1’=’1’ #.

## 2. Επίθεση XSS.

Πατάω logout και μπαίνω με τα στοιχεία χρήστη victim και 123456.

Πηγαίνω στα Notes-announcements και γράφω

Πατάω Submit Note και μου βγάζει αυτό. Δηλαδή έτρεξε Javascript κώδικας στον browser.




Η επίθεση XSS πέτυχε.

### Φάση Άμυνας

Κάνω register με διαφορετικό χρήστη.

---

**New user registration**

secure_user

••••••••••••••••

Register

Ο χρήστης εγγράφηκε επιτυχώς και πλέον ο κωδικός του είναι κρυπτογραφημένος.

| ←T→ | | | id | username | password |
|---|---|---|---|---|---|
| Επεξεργασία | Αντιγραφή | Διαγραφή | 1 | u1 | p1 |
| Επεξεργασία | Αντιγραφή | Διαγραφή | 2 | victim | 123456 |
| Επεξεργασία | Αντιγραφή | Διαγραφή | 3 | secure_user | $2y$10$bn4bKXDolKmorzQUo54TguZDFfv0VNQQbNeaaDnZ0pm... |

*Προστατευμένοι και απροστάτευτοι κωδικοί.*

### 1. Άμυνα SQL injection

Τώρα το SQL injection θα αποτύχει! Αντί η εντολής SQL να παρακάμψει την ασφάλεια, το σύστημα απέκρουσε τη σύνδεση!

**Password Manager**

' OR '1'='1' #

•••••••••

Login

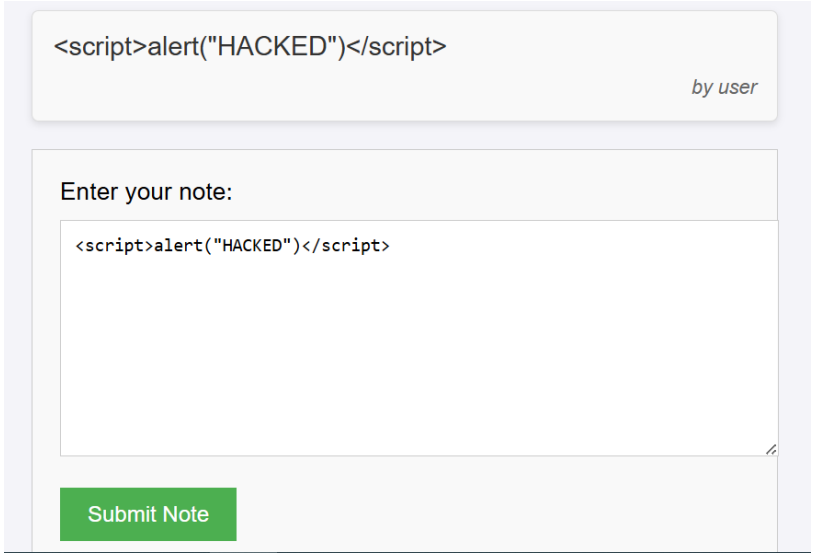
Invalid username or password

Register new user

Home page

## 2. Άμυνα XSS.

Συνδέομαι με κάποιον ήδη registered χρήστη και γράφω στα Notes αυτή την εντολή. Πατάω submit. Παρατηρώ ότι δεν μου βγάζει pop up window αλλά σημειώνει την εντολή στο κουτάκι από πάνω.




Η απόπειρα XSS απέτυχε.